

Subject: Growth Engineer build task — Fady Masoud

Hi,

The client campaign portal is built and deployed. Everything from the submission list is below, in your order.

What's in it

Three brands — Kilele Rides, Karoo Coaches, Marrakech Express — in one portal, 95,176 customers between them. The largest brand is about 90× the smallest, which is what everything was sized against.

- **Dashboard** — total customers, how many are contactable, signups per day over the last 30 days, and how each campaign performed.
- **Contacts** — the full list, searchable per column, 20 to a page.
- **Campaigns** — every campaign with its provider figures beside its event-log figures, and a **View** button through to the detail.
- **Imports** — what loaded, what was refused, and why, in plain English.
- **Sending** — preview, confirm, and a send record that keeps what was approved.
- **Shared link** — one campaign's aggregates, password-protected, for someone with no account.

Every number on screen says how it was counted. Provider-reported figures and event-log counts sit side by side and are never added together, because they disagree — and showing both is more honest than picking one and calling it the truth.

The standard build

1. Live URL

<https://campaign-five-flax.vercel.app>

2. The six logins

Email	Password	Brand	Role
owner@kilele.vg-eval.test	Kilele-Owner-2026	Kilele Rides	owner
analyst@kilele.vg-eval.test	Kilele-Analyst-2026	Kilele Rides	analyst
owner@karoo.vg-eval.test	Karoo-Owner-2026	Karoo Coaches	owner
analyst@karoo.vg-eval.test	Karoo-Analyst-2026	Karoo Coaches	analyst
owner@marrakech.vg-eval.test	Marrakech-Owner-2026	Marrakech Express	owner
analyst@marrakech.vg-eval.test	Marrakech-Analyst-2026	Marrakech Express	analyst

Each lands in its own brand's portal. Owners can send; analysts cannot — and not because the button is hidden. If an analyst writes straight to the database, bypassing the interface entirely, Postgres refuses it.

3. Google sign-in

Live. Sign in with Google from the same login screen and it works for any of the six.

A Google account that is *not* one of the six authenticates successfully and lands on `/no-access` with no data. That is deliberate: signing in proves who somebody is, not that they belong to a brand. Those are separate questions and the portal answers them separately.

4. Supabase

- **Project URL** — `https://hhovyvkphaqniunvjoyw.supabase.co`
- **Anon (publishable) key** — `sb_publishable_Yw4c8cPu7aUuR72DGRXqvA_KccBLcHg`

Which key the deployed app uses: the **publishable** key, everywhere user-facing. The service-role key is server-only and never reaches a browser. The two modules that hold it begin with `import 'server-only'`, so a client component importing them fails the build rather than shipping the secret.

Tables (11)

`brands`, `brand_members`, `contacts`, `campaigns`, `contact_events`, `import_runs`, `import_issues`, `campaign_sends`, `send_recipients`, `send_batches`, `shared_reports`

Functions (16) — 11 callable in `public`, 5 helpers in `app`, which is not exposed through the API

Area	Functions
Isolation and roles	<code>app.current_user_brand_ids()</code> , <code>app.is_brand_owner()</code>
Analytics	<code>public.contactability_breakdown()</code> , <code>public.signups_per_day()</code> , <code>public.campaign_performance()</code> , <code>public.campaign_engagement_detail()</code>
Import	<code>public.import_issue_summary()</code>
Send and provider	<code>public.send_progress()</code> , <code>public.apply_provider_reports()</code> , <code>public.mark_send_recipients()</code>
Shared link	<code>public.unlock_shared_report()</code> , <code>public.hash_report_password()</code>
Security audit	<code>public.security_coverage()</code>
Internal helpers	<code>app.apply_contact_events()</code> , <code>app.delivery_rank()</code> , <code>app.touch_updated_at()</code>

5. The repo

<https://github.com/FadyMasoud/Campaign> — public.

Real history, committed phase by phase as the work happened; it is not a single squashed drop. `schema.sql` is at the root, generated from the 13 migrations. `README.md` is at the root, and `docs/` holds the AI-usage log, a brief-to-build traceability document, and a project handbook.

6. AI tools, time, availability

- **AI tools** — Claude (Opus 5) via Claude Code, used as a pair-programmer. Disclosed phase by phase in `docs/AI-USAGE.md`, including the places I overruled it and why.

- **How long it took** — [fill in]
- **Earliest start date** — [fill in]
- **Notice period** — [fill in]

The send and the shared link

7. Where the send's progress and results are recorded

On screen — `/portal/sends/{id}` : what was approved and by whom, when it went, how many the provider accepted, how many it refused, how it was handed over, where every recipient stands, and a timeline.

In the database

- `campaign_sends` — one row per approved send, with the provider reference.
- `send_recipients` — the frozen audience, one row per person, with delivery status. Frozen at approval, so the record still shows where the message actually went even if the customer later changes their address.
- `send_batches` — one row per call to the provider, because a send of any real size takes more than one.

A real send is on record: **Marrakech** `MAR-0001` — **240 approved, 240 accepted**, provider batch `batch_50e1f5a496627f640a36` . Delivery reports have been pulled back: **223 delivered, 17 bounced**.

Three earlier Kilele sends are also there, showing smaller figures than they approved. That is not a display problem — it is the bug described in the note below, which I found late by reading the stored rows back. The code is fixed and covered by tests; those three rows are left as they happened rather than tidied up, because rewriting a send record to look better is the one thing this portal is built not to do.

8. The provider key

`vgk_c5d8fd95e33712d8c402702a9180543e921ba180022aa084`

You can read the same delivery record I can:

```
curl -H "Authorization: Bearer vgk_c5d8fd95e33712d8c402702a9180543e921ba180022aa084" \  
https://dispatcher-production-72fc.up.railway.app/v1/messages/batch_50e1f5a496627f640a36/events
```

9. The shared link

<https://campaign-five-flax.vercel.app/r/Ls8kBOfYEKahQas6qUnfwchCZY2Hjp8R0knPZsdzAxc>

Password: `velocity-review-2026`

One campaign's aggregates and nothing else. Before the password it shows only the password box — no campaign name, no figures. A guessed token returns not-found with a `noindex` tag rather than confirming anything exists.

10. The note (289 words)

What I tried to break. I replaced the `contacts` RLS policy with `using (true)` and re-ran the suite: five assertions failed and "*sees their own contact*" stayed green, which is what distinguishes isolated from empty. I confirmed a send twice from two sessions at once — one wins, one is refused by a unique index, not a disabled button. As an analyst I inserted straight into `campaign_sends` ; the database refused it. The provider

sent an event whose id was literally `evt-batch_50-forged`, claiming KAROO, naming a KILELE contact, inside a MARRAKECH batch — discarded, because a provider's own label never decides which tenant a row belongs to.

The worst one I found by reading the live data back, not by testing. PostgREST caps every response at 1,000 rows *silently* — no error, just a plausible answer. A Kilele send recorded 23,969 approved while only the first 1,000 were ever frozen or sent to: the portal was the half-sender. Fixing that exposed a second lie — the provider's docs promise 100,000 recipients per call and an empty `rejected` array, but it caps at 500. A send now fans out into as many calls as it needs, each recorded in `send_batches`.

Where isolation lives. `supabase/migrations/20260913160000_brands_and_isolation.sql:104` — `app.current_user_brand_ids()`. Every policy in the project is one line referring to it.

The number I trust least. Karoo's 189 contactable. Correct under the stated rule, but Karoo's customer records claim 483 unsubscribes while the event log holds 4,880 more people who opted out. One source is badly stale, and the portal says so on screen rather than picking.

What isn't finished. Three early Kilele sends still carry the truncated figures, kept rather than rewritten. Delivery reports are polled on demand, not scheduled. Arabic was dropped deliberately.

Thanks for putting this together — it was a genuinely good brief to build against, and the provider behaving nothing like its documentation was the best part of it.

Fady